

Ciencia de Datos

MLOps y seguimiento de experimentos

Del experimento al modelo desplegable:
MLflow, Weights & Biases y DVC con el
caso Wine Quality

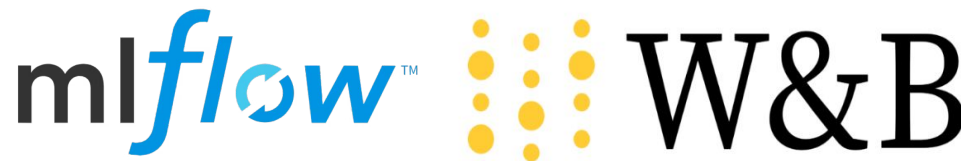


Que es MLops?

MLOps (Machine Learning Operations) es un conjunto de prácticas, herramientas y principios culturales que combinan el Machine Learning (ML), la Ingeniería de Datos y el DevOps tradicional. Su objetivo principal es automatizar, estandarizar y simplificar el ciclo de vida de los modelos de inteligencia artificial, desde su desarrollo inicial hasta su puesta en producción y mantenimiento continuo.

Herramientas a disposición

Las Herramientas principales para la etapa de versionado de datos y modelos son:, MLflow. B&W



Niveles de Madurez en MLops

Nivel 0 (Manual): Todo manual y separado; lento y propenso a errores.

Nivel 1 (Automatización): Entrenamiento continuo y pipelines reproducibles.

Nivel 2 (CI/CD Completa): Despliegues, reentrenamiento automático y monitoreo en producción.

MLflow y W&B: seguimiento y gestión de modelos

Qué problema resuelve

MLflow: Organiza runs, parámetros, métricas y artefactos para rastrear qué configuración produjo cada resultado.

W&B: Plataforma centralizada para seguimiento en tiempo real, visualización interactiva de experimentos y colaboración en la nube.

Quién lo desarrolla

MLflow: Nació en Databricks y se incorporó a la Linux Foundation en 2020; desarrollado por una comunidad abierta.

W&B: Creado por Weights & Biases Inc. (fundada en 2017), orientado a soluciones MLOps para equipos académicos y empresariales.

Licencia

MLflow: Apache 2.0, 100% código abierto y de libre modificación.

W&B: SDK cliente de código abierto (Licencia MIT); plataforma de servidor y servicio web propietarios.

Costo

MLflow: Gratuito sin pago de licencia. El servidor, almacenamiento y respaldos consumen infraestructura propia.

W&B: Servicio gestionado en la nube con plan gratuito (académico/personal) y suscripciones de pago para equipos.

Comparativa rápida: MLflow destaca en arquitecturas autohospedadas y abiertas, mientras que W&B ofrece dashboards en la nube listos para usar.

Ubicación en el ciclo de la ciencia de datos

ETAPA	ACTIVIDAD REALIZADA	APORTE MLOPS
Recopilación	Obtener Wine Quality desde UCI	Conservar fuente e identificador
Limpieza	Revisar faltantes y retirar duplicados	Exportar datos y metadatos
Modelado	Random Forest con diez configuraciones	Registrar parámetros y semilla por run
Evaluación	Selección por validación; prueba aislada	Separar métricas de selección y finales
Visualización	Comparar RMSE, R^2 y tiempos	Consultar CSV y paneles de runs
Despliegue	Empaquetar modelo; prueba de inferencia local	Identificar artefacto, entorno y versión

Instalación y ejemplo mínimo

PowerShell

Prueba sencilla para ver cómo registrar ejecuciones en MLflow. No entrena ningún modelo: los parámetros y las métricas son valores de ejemplo escritos directamente en el código.

```
import mlflow
import sklearn

print("MLflow: ", mlflow.__version__)
print("Scikit-learn: ", sklearn.__version__)

[1] ✓ 7.0s

... MLflow: 3.16.1
     Scikit-learn: 1.9.1

▷ mlflow.set_experiment("mlflow-prueba-minima")

[2] ✓ 2.3s

... <Experiment: artifact_location='/Escritorio/9no/Ciencia de '
    'Datos/MLOps/mlops-wine-quality/notebo/mlruns/1'), creation_time=1791144244516

with mlflow.start_run(run_name="prueba-01") as run:

    mlflow.log_param("modelo", "prueba")
    mlflow.log_param("random_state", 42)

    mlflow.log_metric("rmse", 0.75)
    mlflow.log_metric("r2", 0.82)

    print("Run ID: ", run.info.run_id)

[3] ✓ 1.1s

... Run ID: h4fe0eb04d0ac4dd806e49cb23e1a8c62

print(mlflow.get_tracking_uri())

[4] ✓ 0.0s

... sqlite:///C:/Users/mateo/Documentos/universidad/ciencia%20de%20datos/vinos/mlops-wi
```

Wine Quality: problema, datos y pipeline

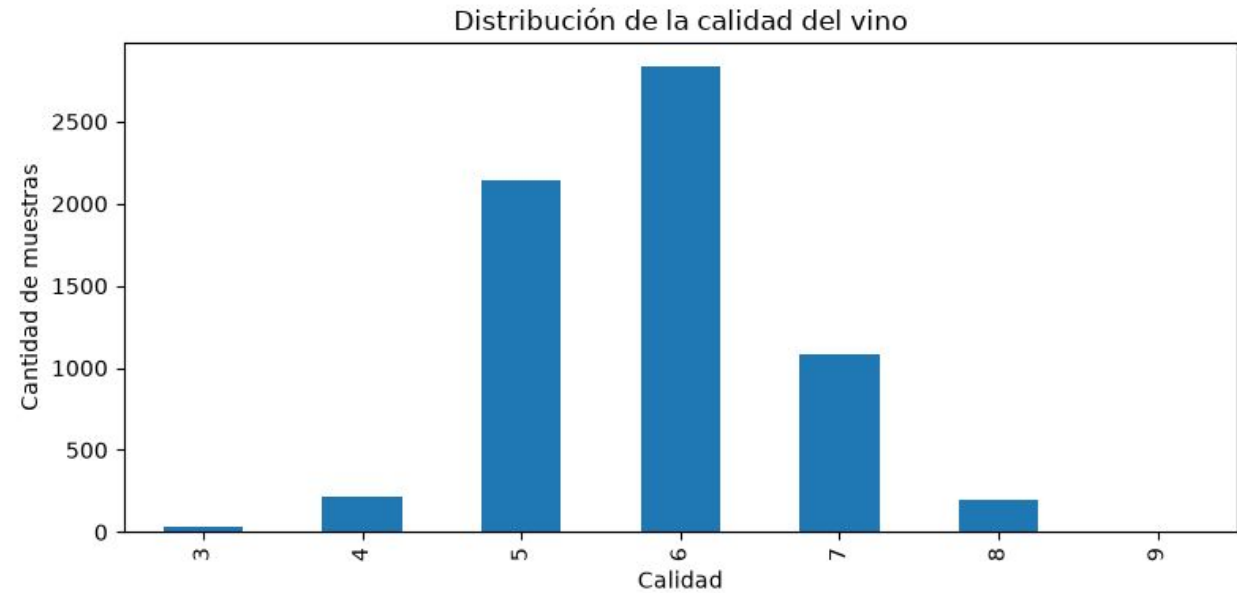
Problema

Regresión: estimar la puntuación sensorial quality a partir de 11 medidas fisicoquímicas.

Tratamiento inicial de Datos

Se procedió a la exploración inicial de columnas, se buscaron valores faltantes, correlación entre variables y depuración de registros duplicados.

Se divide el dataset en entrenamiento, prueba y validación



PIPELINE DE TRABAJO

01
Cargar UCI

02
Limpiar

03
Dividir 60/20/20

04
10 runs Random Forest

05
Prueba final

Control de versiones y trazabilidad del dataset

Para garantizar la **trazabilidad y reproducibilidad** del pipeline, se genera una versión persistente del conjunto de datos después de la etapa de limpieza.

Identificación de la versión

Nombre y versión: Nombre y versión lógica del dataset

Fuente original: UCI Machine Learning Repository

Dimensiones: Número de filas y columnas

Variable objetivo: Variable a predecir definida

Seguridad e integridad: Hash SHA-256 del archivo procesado

Registro en plataformas de MLOps

MLflow

Se almacena su información como **entrada del experimento** y como **artefacto** del pipeline.

Weights & Biases

Se registra como un **Artifact de tipo `dataset`**, permitiendo mantener versiones sucesivas del conjunto de datos.

Diez configuraciones registradas con MLFlow

Mejor configuración (exp. 10)

Se ejecutarán diez configuraciones de Random Forest utilizando el mismo dataset, división de entrenamiento/validación y semilla aleatoria.

El conjunto de prueba permanece aislado durante la selección.

Para cada experimento se registrarán los hiperparámetros, MAE, RMSE y R^2 de validación, además de los tiempos de ejecución.

```
mlflow.set_experiment("wine-quality-random-forest")
```

Experimento 01	RMSE validación=0.7217	R^2 validación=0.3276
Experimento 02	RMSE validación=0.7219	R^2 validación=0.3272
Experimento 03	RMSE validación=0.7225	R^2 validación=0.3260
Experimento 04	RMSE validación=0.6954	R^2 validación=0.3756
Experimento 05	RMSE validación=0.6955	R^2 validación=0.3755
Experimento 06	RMSE validación=0.6945	R^2 validación=0.3772
Experimento 07	RMSE validación=0.6914	R^2 validación=0.3829
Experimento 08	RMSE validación=0.6902	R^2 validación=0.3850
Experimento 09	RMSE validación=0.6891	R^2 validación=0.3869
Experimento 10	RMSE validación=0.6888	R^2 validación=0.3875

Funciones de MLflow aplicadas

```
mlflow.set_experiment()
```

Organiza y agrupa todas las ejecuciones bajo un nombre de proyecto común.

```
mlflow.start_run()
```

Abre un contenedor de registro individual para aislar cada iteración o prueba.

```
mlflow.log_params()
```

Guarda los hiperparámetros de entrada utilizados en el modelo.

```
mlflow.log_metrics()
```

Almacena las métricas de rendimiento (MAE, RMSE, R^2) y tiempos.

```
mlflow.sklearn.log_model()
```

Serializa y guarda el modelo entrenado como un artefacto reutilizable.

Se rankeo a los mejores modelos por RMSE

experimento	n_estimators	max_depth	min_samples_split	min_samples_leaf	mae_validacion	rmse_validacion	r2_validacion	tiempo_entrenamiento
10	300	NaN	5	2	0.535146	0.688779	0.387516	0.694384
9	200	NaN	2	2	0.534300	0.689100	0.386945	0.527782
8	200	NaN	5	1	0.536345	0.690191	0.385001	0.504012
7	100	NaN	2	1	0.536955	0.691357	0.382922	0.290543
6	300	10.0	2	1	0.540310	0.694549	0.377210	0.713205
4	100	10.0	2	1	0.541044	0.695421	0.375645	0.279197
5	200	10.0	2	1	0.540709	0.695530	0.375451	0.547761
1	50	5.0	2	1	0.564188	0.721668	0.327628	0.181371
2	100	5.0	2	1	0.564697	0.721903	0.327189	0.342587
3	200	5.0	2	1	0.564564	0.722546	0.325990	0.575750

Diez configuraciones registradas con W&B

Mejor configuración (exp. 10)

Se repetirán las mismas diez configuraciones utilizadas con MLflow, manteniendo el mismo dataset, división entrenamiento/validación, modelo, semilla y métricas.

El conjunto de prueba permanece aislado hasta la evaluación final.

Esto permite realizar una comparación directa entre ambas herramientas.

```
import wandb

print("W&B:", wandb.__version__)
```

```
wandb: updating run metadata
Experimento 02 | RMSE validación=0.7219 | R² validación=0.3272
wandb: uploading wandb-metadata.json; uploading requirements.txt; uploading wandb
wandb: uploading history steps 0-1, summary
wandb:
wandb: Run history:
wandb:     mae_validacion _
wandb:     r2_validacion _
wandb:     rmse_validacion _
wandb: tiempo_entrenamiento _
wandb: tiempo_total_wandb _
wandb:
wandb: Run summary:
wandb:     mae_validacion 0.5647
wandb:     r2_validacion 0.32719
wandb:     rmse_validacion 0.7219
wandb: tiempo_entrenamiento 0.35158
wandb: tiempo_total_wandb 0.35849
wandb:
```

Funciones de W&B aplicadas

`wandb.init() :`

Inicia la ejecución y registra el proyecto y los hiperparámetros.

`run.log() :`

`run.log()`: Envía las métricas de rendimiento y tiempos al panel de control.

`run.id:`

Obtiene el identificador único de la ejecución actual.

Tras la ejecución se encuentra que los parámetros del experimento 10 son los que mejor evaluación de RMSE obtuvieron. Adicionalmente estos son los mismos que los señalados por MLflow.

Benchmark de sobrecarga de las herramientas

Mejor configuración (exp. 10)

Con la configuración ganadora se realizan cinco repeticiones para cada condición: sin tracking, MLflow local y W&B offline. Se mide con `time.perf_counter()` el entrenamiento, la predicción y las métricas; con tracking se incluye también el contexto de inicio y finalización del run. No se serializan modelos en este benchmark.

$$100 \left(\frac{\text{mediana}(t_{\text{tracking}})}{\text{mediana}(t_{\text{sin tracking}})} - 1 \right).$$

Condición	Mediana (s)	Sobrecarga
Sin tracking	1.658416	0 %
MLflow local	1.813136	9.33 %
W&B offline	2.140149	29.05 %

Poner un modelo en producción

Una vez identificado y reproducido el mejor experimento, la configuración seleccionada se reentrena con todo el conjunto de desarrollo.

Ese modelo final se registra en MLflow y como artefacto en Weights & Biases.

Este registro se realiza de forma independiente a los diez experimentos para no alterar la comparación del rendimiento entre configuraciones.

```
ruta_modelo = DIR_MODELOS / "random_forest_mejor.joblib"

ruta_modelo.parent.mkdir(
    parents=True,
    exist_ok=True
)

joblib.dump(
    modelo_final,
    ruta_modelo
)

print(ruta_modelo)
```

Errores encontrados y solución aplicada

Incidencia	Solución y resultado
Python no iniciaba en el entorno aislado.	Se utilizó el intérprete fuera de ese aislamiento y se creó el entorno virtual del proyecto.
MLflow no encontraba uvicorn.	Se añadió .venv/Scripts a PATH, la lista de carpetas donde se buscan programas. El reintento HTTP funcionó.
W&B dejó un proceso abierto.	Se cerró el proceso identificado como propio de la ejecución. El ejecutor incorpora cierre explícito para futuras corridas.

Una empresa energética europea gestiona cientos de modelos de la red eléctrica con MLflow

Notebooks para desarrollar, MLflow Tracking para métricas y trabajos de Databricks para inferencia, con un proceso común para muchos modelos y bibliotecas.

La publicación no identifica la empresa ni cuantifica ahorros: no se atribuyen porcentajes. Fuente: Zaharia et al. (2018, p. 43).

Referencias, 1 de 2

Cortez, P., Cerdeira, A., Almeida, F., Matos, T., & Reis, J. (2009). *Wine quality* [Conjunto de datos]. UCI Machine Learning Repository.

<https://doi.org/10.24432/C56S3T>

DVC. (s. f.-a). *Data versioning and ML experiments* [Software]. GitHub. Recuperado el 6 de octubre de 2026, de <https://github.com/treeverse/dvc>

DVC. (s. f.-b). *What is DVC?* Recuperado el 6 de octubre de 2026, de <https://doc.dvc.org/user-guide>

Linux Foundation. (2020, 25 de junio). *The MLflow project joins Linux Foundation*.

<https://www.linuxfoundation.org/press/press-release/the-mlflow-project-joins-linux-foundation>

Macas, B., & Sangurima, J. (2026). *Pipeline MLOps para Wine Quality* [Repositorio de código y resultados]. GitHub.

<https://github.com/bryzcoll/mlops-wine-quality>

Referencias, 2 de 2

MLflow. (s. f.-a). *Deploy MLflow model as a local inference server*. Recuperado el 6 de octubre de 2026, de <https://mlflow.org/docs/latest/ml/deployment/deploy-model-locally/>

MLflow. (s. f.-b). *MLflow* [Software]. GitHub. Recuperado el 6 de octubre de 2026, de <https://github.com/mlflow/mlflow>

Weights & Biases. (s. f.-a). *Explore Weights & Biases pricing plans*. Recuperado el 6 de octubre de 2026, de <https://site.wandb.ai/pricing/>

Weights & Biases. (s. f.-b). *Weights & Biases* [SDK de Python]. GitHub. Recuperado el 6 de octubre de 2026, de <https://github.com/wandb/wandb>

Zaharia, M., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Konwinski, A., Murching, S., Nykodym, T., Ogilvie, P., Parkhe, M., Xie, F., & Zumar, C. (2018). Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4), 39–45. <https://www.databricks.com/sites/default/files/2023-08/accelerating-the-machine-learning-lifecycle-with-mlflow.pdf>

**Botaron al Ingeniero y se llevó los modelos
entrenados en su máquina**

